Hubei University

Academic Year 2025 -- 2026

Semester 2

Student Laboratory Report Book

Institute：Manchester United College

Student Name：Qin Tian

Class：2402

Student ID：202431123002054

Course Course：Big Data Analysis

Instructor：Li Jie

Student Laboratory Rules

1. Students must conduct experiments at the scheduled time and may not be absent or late without cause.

2. Before each experiment, students must preview the assigned experiment and prepare a preview report as required.

3. Before each experiment, students must submit the previous experiment report and the preview report for the current experiment. The experiment may begin only after the instructor has checked and approved them.

4. After reaching their assigned lab station, students must check the instruments they use against the equipment list for any missing or damaged items. Any problems must be reported to the instructor promptly. Using another group's instruments without authorization is strictly forbidden.

5. During the experiment, students must maintain a rigorous scientific attitude and strictly follow instrument operating procedures and precautions.

6. After the experiment, students must submit their data to the instructor for inspection. Once approved, they should restore and

organize the instruments and equipment before leaving the lab.

7. Keep the laboratory quiet and tidy. Loud talking, littering, and eating are prohibited.

8. If an accident or damage to instruments or equipment is caused by violating operating procedures or conducting experiments without permission, it must be reported promptly. A damage list must be completed, and compensation shall be made according to the college's relevant regulations.

Big Data Analysis and Application Experiment 1

Laboratory: Innovation & Entrepreneurship 604 Date: 2026.3.20

1.Problem Description

This experiment uses **employee attrition data** from **pfm_train.csv** and **pfm_test.csv** to complete a **full workflow** from **data preparation** to **linear-model prediction**. The training set contains the **target variable** Attrition, while the test set only contains **employee features**, so the task is not only to **clean the raw data**, but also to make sure the processed training set and test set can be used in the **same modeling pipeline**.

At the beginning, the dataset looked simple, but after checking it carefully I found that the real difficulty was not just **"running a model."** Before the linear model could be trained, I first needed to **understand the structure of the data**, identify columns that were **useless for prediction**, deal with **categorical variables** that could not be fed directly into a model, and keep the **feature columns consistent** between training and test data. After that, I also needed to use the prepared data to **build and compare several models**, and then observe whether the **preprocessing** really helped the later **prediction task**.

So the main problem in this experiment was how to turn **raw employee data** into a **clean, consistent, and model-ready dataset**, and then use that dataset to carry out a **reasonable linear-model practice**. In other words, this experiment was not only about **cleaning data**, but also about connecting **preprocessing**, **visualization**, **feature preparation**, **model training**, and **result checking** into one **complete process**.

2. Basic Approach

My basic idea in this experiment was to move **step by step** instead of jumping directly into **modeling**. First, I used **pandas** to read the two csv files and checked their **shape**, **data types**, **missing values**, **duplicate rows**, and **constant columns**. This helped me understand what problems really existed in the dataset, because if I skipped this step and directly trained a model,

it would be easy to ignore **hidden issues in the raw data**.

After the inspection, I removed **Over18**, **StandardHours**, and **EmployeeNumber**. The first two columns had almost no **useful variation**, and EmployeeNumber was only an **identifier**, so keeping them would not help the **prediction task**. Then I focused on the **categorical variables**. Since **linear models** cannot use **text features** directly, I applied **pd.get_dummies()** to convert categorical columns into **numerical dummy variables**. At the same time, I split the training data into **X_train** and **y_train**, and used **reindex()** to make the test set follow exactly the same **feature columns** as the training set. This step was very important, because even if the **encoding method** is correct, the model will still fail or give **unreliable results** if the train and test features are **not aligned**.

Next, I standardized the original **numeric features** with **StandardScaler**. I fit the scaler only on the **training data** and then applied the same transformation to the **test data**, so the preprocessing process stayed **consistent** and avoided **data leakage**. After that, I carried out a small **modeling stage** on the processed data. I split the training data again into a **fitting set** and a **validation set**, then trained several models, including **logistic regression** with different **regularization settings**, **SGDClassifier** with **log-loss**, and a **random forest baseline** for comparison. Finally, I compared the models with metrics such as **accuracy**, **precision**, **recall**, **F1-score**, and **ROC-AUC**, and also drew several figures such as **ROC curves**, **confusion matrices**, and **coefficient plots** to make the results clearer.

3. Source Code

3.1 Read the raw csv files

This step reads the two csv files and takes a quick look at the dataset size and the first few rows.

```python
# use pandas to read the csv files
import pandas as pd

# this one is for scaling the numeric features later
from sklearn.preprocessing import StandardScaler

# read train data and test data
train = pd.read_csv("pfm_train.csv")
test = pd.read_csv("pfm_test.csv")

# just check the size first
print("train shape:", train.shape)
print("test shape:", test.shape)

# have a look of the first few rows
train.head()
```

[10] ✓ 0.0s

```python
# check basic info of train set
print("=== train info ===")
train.info()

# also check the test set
print("\n=== test info ===")
test.info()

# see if there is any missing values in train
print("\n=== missing values in train ===")
print(train.isnull().sum())

# same thing for test
print("\n=== missing values in test ===")
print(test.isnull().sum())
```

1] ✓ 0.0s

```python
# check duplicate rows first
print("duplicate rows in train:", train.duplicated().sum())
print("duplicate rows in test:", test.duplicated().sum())

# find the columns which only have one value
const_train = [col for col in train.columns if train[col].nunique(dropna=False) == 1]
const_test = [col for col in test.columns if test[col].nunique(dropna=False) == 1]

# print them out
print("constant cols in train:", const_train)
print("constant cols in test:", const_test)
```

[12] ✓ 0.1s

```python
# see the target label distribution
print("attrition counts:")
print(train["Attrition"].value_counts())

# find all categorical columns in raw data
cat_cols_raw = train.select_dtypes(include="object").columns.tolist()
print("\nraw categorical cols:", cat_cols_raw)

# check each category values
for col in cat_cols_raw:
    print(f"\nvalue counts of {col}:")
    print(train[col].value_counts())
```

13] ✓ 0.0s

```python
# copy the data first, so the raw one not be changed
train_clean = train.copy()
test_clean = test.copy()

# these columns are not useful, so remove them
# Over18 and StandardHours are basically same value
# EmployeeNumber is just id
drop_cols = ["Over18", "StandardHours", "EmployeeNumber"]

# drop these columns in both train and test
train_clean = train_clean.drop(columns=drop_cols)
test_clean = test_clean.drop(columns=drop_cols)

# check the shape after dropping
print("train_clean shape:", train_clean.shape)
print("test_clean shape:", test_clean.shape)
```

[14] ✓ 0.0s

```python
# get the categorical columns after dropping useless ones
cat_cols = train_clean.select_dtypes(include="object").columns.tolist()
print("categorical cols after drop:", cat_cols)

# do one-hot encoding for categorical features
# drop_first=True to avoid some repeated dummy columns
train_encoded = pd.get_dummies(train_clean, columns=cat_cols, drop_first=True)
test_encoded = pd.get_dummies(test_clean, columns=cat_cols, drop_first=True)

# split x and y from training data
X_train = train_encoded.drop(columns="Attrition")
y_train = train_encoded["Attrition"]

# make test columns same with train columns
# if some column is missing in test, fill it by 0
X_test = test_encoded.reindex(columns=X_train.columns, fill_value=0)

# find bool type columns
bool_cols_train = X_train.select_dtypes(include="bool").columns
bool_cols_test = X_test.select_dtypes(include="bool").columns

# change bool to int, maybe better for model later
X_train[bool_cols_train] = X_train[bool_cols_train].astype(int)
X_test[bool_cols_test] = X_test[bool_cols_test].astype(int)

# check shape after encoding
print("encoded train shape:", train_encoded.shape)
print("encoded test shape:", test_encoded.shape)
print("X_train shape:", X_train.shape)
print("X_test shape:", X_test.shape)
```

5] ✓ 0.0s Python

```python
# pick numeric columns for scaling
# do not include the target column
num_cols = train_clean.select_dtypes(exclude="object").columns.tolist()
num_cols.remove("Attrition")

# print these numeric columns
print("scaled numeric cols:", num_cols)

# create the scaler
scaler = StandardScaler()

# fit on train data first, then transform it
X_train[num_cols] = scaler.fit_transform(X_train[num_cols])

# use the same scaler on test data
X_test[num_cols] = scaler.transform(X_test[num_cols])

# look the processed train data
X_train.head()
```

✓ 0.0s Python

This step checks the final shape of the processed data and confirms that no missing value is left.

```python
# final check of the shapes
print("final X_train shape:", X_train.shape)
print("final y_train shape:", y_train.shape)
print("final X_test shape:", X_test.shape)

# see if there is still missing values
print("missing values in X_train:", X_train.isnull().sum().sum())
print("missing values in X_test:", X_test.isnull().sum().sum())

# check the final data types
print("\nX_train dtype summary:")
print(X_train.dtypes.value_counts())
```

17] ✓ 0.0s

```python
import numpy as np
import pandas as pd
from pathlib import Path

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression, SGDClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    roc_curve,
    confusion_matrix,
)

sns.set_theme(style="whitegrid", context="talk")
plt.rcParams["figure.figsize"] = (10, 6)
plt.rcParams["axes.titlesize"] = 16
plt.rcParams["axes.labelsize"] = 13

if "train" not in globals():
    train = pd.read_csv("pfm_train.csv")

if "X_train" in globals() and "y_train" in globals():
    prepared_X = X_train.copy()
    prepared_y = pd.Series(y_train).copy()
else:
    train_clean = train.drop(columns=["Over18", "StandardHours", "EmployeeNumber"]).copy()
    cat_cols = train_clean.select_dtypes(include="object").columns.tolist()
    train_encoded = pd.get_dummies(train_clean, columns=cat_cols, drop_first=True)
    prepared_X = train_encoded.drop(columns="Attrition")
    prepared_y = train_encoded["Attrition"]
    bool_cols = prepared_X.select_dtypes(include="bool").columns
    prepared_X[bool_cols] = prepared_X[bool_cols].astype(int)

prepared_y = pd.Series(prepared_y, name="Attrition")

X_fit, X_valid, y_fit, y_valid = train_test_split(
    prepared_X,
    prepared_y,
    test_size=0.25,
    random_state=42,
    stratify=prepared_y,
)

fig_dir = Path("experiment1_figures")
fig_dir.mkdir(exist_ok=True)

print("prepared_X shape:", prepared_X.shape)
print("prepared_y shape:", prepared_y.shape)
print("X_fit shape:", X_fit.shape)
print("X_valid shape:", X_valid.shape)
print("validation label distribution:")
print(y_valid.value_counts(normalize=True).round(4))
```

```python
fig, axes = plt.subplots(2, 2, figsize=(18, 13))

sns.countplot(data=train, x="Attrition", hue="Attrition", palette="Set2", ax=axes[0, 0], legend=False)
axes[0, 0].set_title("Attrition Label Distribution")
axes[0, 0].set_xlabel("Attrition")
axes[0, 0].set_ylabel("Count")

sns.countplot(data=train, x="OverTime", hue="Attrition", palette="Set1", ax=axes[0, 1])
axes[0, 1].set_title("OverTime vs Attrition")
axes[0, 1].set_xlabel("OverTime")
axes[0, 1].set_ylabel("Count")

sns.histplot(data=train, x="Age", hue="Attrition", kde=True, bins=20, palette="Set2", ax=axes[1, 0], element="step")
axes[1, 0].set_title("Age Distribution by Attrition")
axes[1, 0].set_xlabel("Age")
axes[1, 0].set_ylabel("Frequency")

sns.boxplot(data=train, x="Attrition", y="MonthlyIncome", hue="Attrition", palette="Pastel1", ax=axes[1, 1], legend=False)
axes[1, 1].set_title("Monthly Income by Attrition")
axes[1, 1].set_xlabel("Attrition")
axes[1, 1].set_ylabel("Monthly Income")

plt.tight_layout()
```

```python
fig, axes = plt.subplots(1, 3, figsize=(22, 7))

department_rate = (
    train.groupby("Department")["Attrition"]
    .mean()
    .sort_values(ascending=False)
    .mul(100)
    .round(2)
)
sns.barplot(
    x=department_rate.values,
    y=department_rate.index,
    palette="viridis",
    ax=axes[0],
)
axes[0].set_title("Attrition Rate by Department")
axes[0].set_xlabel("Attrition Rate (%)")
axes[0].set_ylabel("Department")

jobrole_rate = (
    train.groupby("JobRole")["Attrition"]
    .mean()
    .sort_values(ascending=False)
    .mul(100)
    .round(2)
)
sns.barplot(
    x=jobrole_rate.values,
    y=jobrole_rate.index,
    palette="magma",
    ax=axes[1],
)
axes[1].set_title("Attrition Rate by Job Role")
axes[1].set_xlabel("Attrition Rate (%)")
axes[1].set_ylabel("Job Role")

education_rate = (
    train.groupby("EducationField")["Attrition"]
    .mean()
    .sort_values(ascending=False)
    .mul(100)
    .round(2)
)
sns.barplot(
    x=education_rate.values,
    y=education_rate.index,
    palette="crest",
    ax=axes[2],
)
axes[2].set_title("Attrition Rate by Education Field")
axes[2].set_xlabel("Attrition Rate (%)")
axes[2].set_ylabel("Education Field")

plt.tight_layout()
fig.savefig(fig_dir / "figure_02_attrition_rate_by_category.png", dpi=200, bbox_inches="tight")
plt.show()
```

2.0s

```
\Users\rober\AppData\Local\Temp\ipykernel_30896\1488354625.py:10: FutureWarning:

ssing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` var
```

```python
corr_source = train.select_dtypes(exclude="object").copy()
corr_target = corr_source.corr(numeric_only=True)["Attrition"].drop("Attrition")
top_corr_features = corr_target.abs().sort_values(ascending=False).head(10).index.tolist()
heatmap_columns = top_corr_features + ["Attrition"]

plt.figure(figsize=(12, 9))
sns.heatmap(
    corr_source[heatmap_columns].corr(),
    annot=True,
    fmt=".2f",
    cmap="RdBu_r",
    center=0,
    square=True,
)
plt.title("Correlation Heatmap of the Most Relevant Numeric Features")
plt.tight_layout()
plt.savefig(fig_dir / "figure_03_correlation_heatmap.png", dpi=200, bbox_inches="tight")
plt.show()

print("Top numeric correlations with Attrition:")
print(corr_target.sort_values(key=lambda s: s.abs(), ascending=False).head(10))
```

```python
models = {
    "Logistic Regression (L2)": LogisticRegression(
        max_iter=2000,
        class_weight="balanced",
        random_state=42,
    ),
    "Logistic Regression (L1)": LogisticRegression(
        max_iter=3000,
        class_weight="balanced",
        penalty="l1",
        solver="liblinear",
        random_state=42,
    ),
    "SGD Log-Loss": SGDClassifier(
        loss="log_loss",
        class_weight="balanced",
        max_iter=3000,
        tol=1e-4,
        random_state=42,
    ),
    "Random Forest": RandomForestClassifier(
        n_estimators=400,
        min_samples_leaf=2,
        class_weight="balanced",
        random_state=42,
    ),
}

model_results = []
fitted_models = {}

for name, model in models.items():
    model.fit(X_fit, y_fit)
    fitted_models[name] = model

    y_pred = model.predict(X_valid)
    if hasattr(model, "predict_proba"):
        y_score = model.predict_proba(X_valid)[:, 1]
    else:
        raw_score = model.decision_function(X_valid)
        y_score = 1 / (1 + np.exp(-raw_score))

    model_results.append(
        {
            "Model": name,
            "Accuracy": accuracy_score(y_valid, y_pred),
            "Precision": precision_score(y_valid, y_pred),
            "Recall": recall_score(y_valid, y_pred),
            "F1": f1_score(y_valid, y_pred),
            "ROC_AUC": roc_auc_score(y_valid, y_score),
        }
    )

metrics_df = pd.DataFrame(model_results).sort_values(
    by=["ROC_AUC", "F1"],
    ascending=False,
).reset_index(drop=True)

metrics_df.to_csv("model_metrics.csv", index=False)
metrics_df.style.format(
    {
        "Accuracy": "{:.4f}",
        "Precision": "{:.4f}",
        "Recall": "{:.4f}",
        "F1": "{:.4f}",
        "ROC_AUC": "{:.4f}",
    }
)
```

✓ 2.6s

```
I:\kaifagongju\Anaconda\envs\DL\Lib\site-packages\sklearn\linear_model\_logistic.py:1135: FutureWarning:
  warnings.warn(
I:\kaifagongju\Anaconda\envs\DL\Lib\site-packages\sklearn\linear_model\_logistic.py:1160: UserWarning: In
```

```python
plt.figure(figsize=(11, 8))

for name, model in fitted_models.items():
    if hasattr(model, "predict_proba"):
        y_score = model.predict_proba(X_valid)[:, 1]
    else:
        raw_score = model.decision_function(X_valid)
        y_score = 1 / (1 + np.exp(-raw_score))

    fpr, tpr, _ = roc_curve(y_valid, y_score)
    auc_value = roc_auc_score(y_valid, y_score)
    plt.plot(fpr, tpr, linewidth=2.5, label=f"{name} (AUC={auc_value:.3f})")

plt.plot([0, 1], [0, 1], linestyle="--", color="gray", linewidth=1.5, label="Random Guess")
plt.title("ROC Curves of Different Models")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.legend(loc="lower right")
plt.tight_layout()
plt.savefig(fig_dir / "figure_04_roc_curves.png", dpi=200, bbox_inches="tight")
plt.show()
```

✓ 1.2s

```
top_positive = log_coef.tail(8).sort_values(ascending=False)

rf_importance = (
    pd.Series(rf_model.feature_importances_, index=prepared_X.columns)
    .sort_values(ascending=False)
    .head(12)
)

top_positive.to_csv("logistic_top_positive_coefficients.csv", header=["coefficient"])
top_negative.to_csv("logistic_top_negative_coefficients.csv", header=["coefficient"])
rf_importance.to_csv("random_forest_feature_importances.csv", header=["importance"])

fig, axes = plt.subplots(1, 2, figsize=(18, 8))

coef_plot = pd.concat([top_positive, top_negative]).sort_values()
sns.barplot(
    x=coef_plot.values,
    y=coef_plot.index,
    palette=["#4c78a8" if value < 0 else "#e45756" for value in coef_plot.values],
    ax=axes[0],
)
axes[0].set_title("Most Influential Logistic Regression Coefficients")
axes[0].set_xlabel("Coefficient Value")
axes[0].set_ylabel("Feature")

sns.barplot(
    x=rf_importance.values,
    y=rf_importance.index,
    palette="rocket",
    ax=axes[1],
)
axes[1].set_title("Top Random Forest Feature Importances")
axes[1].set_xlabel("Importance")
axes[1].set_ylabel("Feature")

plt.tight_layout()
fig.savefig(fig_dir / "figure_06_model_interpretation.png", dpi=200, bbox_inches="tight")
plt.show()

print("Top positive logistic coefficients:")
print(top_positive)
print()
print("Top negative logistic coefficients:")
print(top_negative)
```

3.0s

```
\Users\rober\AppData\Local\Temp\ipykernel_30896\3340413172.py:21: FutureWarning:

ssing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to

sns.barplot(
\Users\rober\AppData\Local\Temp\ipykernel_30896\3340413172.py:31: FutureWarning:
```

```
######################################################################
# Part 1. Read the raw csv files
######################################################################


# use pandas to read the csv files
import matplotlib
```

```
import numpy as np
import pandas as pd
import seaborn as sns
from pathlib import Path

from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression, SGDClassifier
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    f1_score,
    precision_score,
    recall_score,
    roc_auc_score,
    roc_curve,
)
from sklearn.model_selection import train_test_split

# this one is for scaling the numeric features later
from sklearn.preprocessing import StandardScaler

# use a non-interactive backend so figures can be saved by script
matplotlib.use("Agg")
import matplotlib.pyplot as plt

# read train data and test data
train = pd.read_csv("pfm_train.csv")
test = pd.read_csv("pfm_test.csv")

# just check the size first
print("train shape:", train.shape)
print("test shape:", test.shape)

# have a look of the first few rows
train.head()


##################################################################
# Part 2. Check basic information and missing values
##################################################################
```

```
# check basic info of train set
print("=== train info ===")
train.info()

# also check the test set
print("\n=== test info ===")
test.info()

# see if there is any missing values in train
print("\n=== missing values in train ===")
print(train.isnull().sum())

# same thing for test
print("\n=== missing values in test ===")
print(test.isnull().sum())


################################################################
# Part 3. Check duplicate rows and constant columns
################################################################

# check duplicate rows first
print("duplicate rows in train:", train.duplicated().sum())
print("duplicate rows in test:", test.duplicated().sum())

# find the columns which only have one value
const_train = [col for col in train.columns if
train[col].nunique(dropna=False) == 1]
const_test = [col for col in test.columns if test[col].nunique(dropna=False)
== 1]

# print them out
print("constant cols in train:", const_train)
print("constant cols in test:", const_test)


################################################################
# Part 4. Check the label and categorical columns
################################################################

# see the target label distribution
```

```
print("attrition counts:")
print(train["Attrition"].value_counts())

# find all categorical columns in raw data
cat_cols_raw = train.select_dtypes(include="object").columns.tolist()
print("\nraw categorical cols:", cat_cols_raw)

# check each category values
for col in cat_cols_raw:
    print(f"\nvalue counts of {col}:")
    print(train[col].value_counts())


################################################################
# Part 5. Drop useless columns
################################################################

# copy the data first, so the raw one not be changed
train_clean = train.copy()
test_clean = test.copy()

# these columns are not useful, so remove them
# Over18 and StandardHours are basically same value
# EmployeeNumber is just id
drop_cols = ["Over18", "StandardHours", "EmployeeNumber"]

# drop these columns in both train and test
train_clean = train_clean.drop(columns=drop_cols)
test_clean = test_clean.drop(columns=drop_cols)

# check the shape after dropping
print("train_clean shape:", train_clean.shape)
print("test_clean shape:", test_clean.shape)


################################################################
# Part 6. Encode categorical features
################################################################

# get the categorical columns after dropping useless ones
cat_cols = train_clean.select_dtypes(include="object").columns.tolist()
```

```
print("categorical cols after drop:", cat_cols)

# do one-hot encoding for categorical features
# drop_first=True to avoid some repeated dummy columns
train_encoded = pd.get_dummies(train_clean, columns=cat_cols,
drop_first=True)
test_encoded = pd.get_dummies(test_clean, columns=cat_cols,
drop_first=True)

# split x and y from training data
X_train = train_encoded.drop(columns="Attrition")
y_train = train_encoded["Attrition"]

# make test columns same with train columns
# if some column is missing in test, fill it by 0
X_test = test_encoded.reindex(columns=X_train.columns, fill_value=0)

# find bool type columns
bool_cols_train = X_train.select_dtypes(include="bool").columns
bool_cols_test = X_test.select_dtypes(include="bool").columns

# change bool to int, maybe better for model later
X_train[bool_cols_train] = X_train[bool_cols_train].astype(int)
X_test[bool_cols_test] = X_test[bool_cols_test].astype(int)

# check shape after encoding
print("encoded train shape:", train_encoded.shape)
print("encoded test shape:", test_encoded.shape)
print("X_train shape:", X_train.shape)
print("X_test shape:", X_test.shape)


################################################################
# Part 7. Scale numeric features
################################################################

# pick numeric columns for scaling
# do not include the target column
num_cols = train_clean.select_dtypes(exclude="object").columns.tolist()
num_cols.remove("Attrition")
```

```
# print these numeric columns
print("scaled numeric cols:", num_cols)

# create the scaler
scaler = StandardScaler()

# fit on train data first, then transform it
X_train[num_cols] = scaler.fit_transform(X_train[num_cols])

# use the same scaler on test data
X_test[num_cols] = scaler.transform(X_test[num_cols])

# look the processed train data
X_train.head()


############################################################
# Part 8. Final check of prepared data
############################################################

# final check of the shapes
print("final X_train shape:", X_train.shape)
print("final y_train shape:", y_train.shape)
print("final X_test shape:", X_test.shape)

# see if there is still missing values
print("missing values in X_train:", X_train.isnull().sum().sum())
print("missing values in X_test:", X_test.isnull().sum().sum())

# check the final data types
print("\nX_train dtype summary:")
print(X_train.dtypes.value_counts())


############################################################
# Part 9. Prepare the final modeling matrix
############################################################

# save the preprocessing output for the later linear-model practice
# always write to the sibling experiment folder, so the files can be
regenerated anytime
```

```
output_dir = Path.cwd().parent / "实验二"
output_dir.mkdir(parents=True, exist_ok=True)

X_train.to_csv(output_dir / "X_train_preprocessed.csv", index=False)
y_train.to_frame(name="Attrition").to_csv(output_dir /
"y_train_preprocessed.csv", index=False)
X_test.to_csv(output_dir / "X_test_preprocessed.csv", index=False)

print("saved:", output_dir / "X_train_preprocessed.csv")
print("saved:", output_dir / "y_train_preprocessed.csv")
print("saved:", output_dir / "X_test_preprocessed.csv")


##################################################################
# Part 10. Prepare validation data and figure folder
##################################################################

sns.set_theme(style="whitegrid", context="talk")
plt.rcParams["figure.figsize"] = (10, 6)
plt.rcParams["axes.titlesize"] = 16
plt.rcParams["axes.labelsize"] = 13

X_fit, X_valid, y_fit, y_valid = train_test_split(
    X_train,
    y_train,
    test_size=0.25,
    random_state=42,
    stratify=y_train,
)

figure_dir = Path.cwd() / "experiment1_figures"
figure_dir.mkdir(exist_ok=True)

print("X_fit shape:", X_fit.shape)
print("X_valid shape:", X_valid.shape)
print("validation label distribution:")
print(y_valid.value_counts(normalize=True).round(4))


##################################################################
# Part 11. Draw and save the main figures
```

```
#############################################################

fig, axes = plt.subplots(2, 2, figsize=(18, 13))

sns.countplot(data=train, x="Attrition", hue="Attrition", palette="Set2",
ax=axes[0, 0], legend=False)
axes[0, 0].set_title("Attrition Label Distribution")
axes[0, 0].set_xlabel("Attrition")
axes[0, 0].set_ylabel("Count")

sns.countplot(data=train, x="OverTime", hue="Attrition", palette="Set1",
ax=axes[0, 1])
axes[0, 1].set_title("OverTime vs Attrition")
axes[0, 1].set_xlabel("OverTime")
axes[0, 1].set_ylabel("Count")

sns.histplot(data=train, x="Age", hue="Attrition", kde=True, bins=20,
palette="Set2", ax=axes[1, 0], element="step")
axes[1, 0].set_title("Age Distribution by Attrition")
axes[1, 0].set_xlabel("Age")
axes[1, 0].set_ylabel("Frequency")

sns.boxplot(data=train, x="Attrition", y="MonthlyIncome",
hue="Attrition", palette="Pastel1", ax=axes[1, 1], legend=False)
axes[1, 1].set_title("Monthly Income by Attrition")
axes[1, 1].set_xlabel("Attrition")
axes[1, 1].set_ylabel("Monthly Income")

plt.tight_layout()
fig.savefig(figure_dir / "figure_01_label_and_basic_patterns.png",
dpi=200, bbox_inches="tight")
plt.close(fig)

fig, axes = plt.subplots(1, 3, figsize=(22, 7))

department_rate = (
    train.groupby("Department")["Attrition"]
    .mean()
    .sort_values(ascending=False)
    .mul(100)
```

```
    .round(2)
)
sns.barplot(
    x=department_rate.values,
    y=department_rate.index,
    palette="viridis",
    ax=axes[0],
)
axes[0].set_title("Attrition Rate by Department")
axes[0].set_xlabel("Attrition Rate (%)")
axes[0].set_ylabel("Department")

jobrole_rate = (
    train.groupby("JobRole")["Attrition"]
    .mean()
    .sort_values(ascending=False)
    .mul(100)
    .round(2)
)
sns.barplot(
    x=jobrole_rate.values,
    y=jobrole_rate.index,
    palette="magma",
    ax=axes[1],
)
axes[1].set_title("Attrition Rate by Job Role")
axes[1].set_xlabel("Attrition Rate (%)")
axes[1].set_ylabel("Job Role")

education_rate = (
    train.groupby("EducationField")["Attrition"]
    .mean()
    .sort_values(ascending=False)
    .mul(100)
    .round(2)
)
sns.barplot(
    x=education_rate.values,
    y=education_rate.index,
    palette="crest",
```

```
    ax=axes[2],
)
axes[2].set_title("Attrition Rate by Education Field")
axes[2].set_xlabel("Attrition Rate (%)")
axes[2].set_ylabel("Education Field")

plt.tight_layout()
fig.savefig(figure_dir / "figure_02_attrition_rate_by_category.png",
dpi=200, bbox_inches="tight")
plt.close(fig)

corr_source = train.select_dtypes(exclude="object").copy()
corr_target =
corr_source.corr(numeric_only=True)["Attrition"].drop("Attrition")
top_corr_features =
corr_target.abs().sort_values(ascending=False).head(10).index.tolist()
heatmap_columns = top_corr_features + ["Attrition"]

fig, ax = plt.subplots(figsize=(12, 9))
sns.heatmap(
    corr_source[heatmap_columns].corr(),
    annot=True,
    fmt=".2f",
    cmap="RdBu_r",
    center=0,
    square=True,
    ax=ax,
)
ax.set_title("Correlation Heatmap of the Most Relevant Numeric Features")
plt.tight_layout()
fig.savefig(figure_dir / "figure_03_correlation_heatmap.png", dpi=200,
bbox_inches="tight")
plt.close(fig)

print("top numeric correlations with Attrition:")
print(corr_target.sort_values(key=lambda s: s.abs(),
ascending=False).head(10))


##############################################################
# Part 12. Train multiple models and save the metrics
```

```
################################################################

models = {
    "Logistic Regression (L2)": LogisticRegression(
        max_iter=2000,
        class_weight="balanced",
        random_state=42,
    ),
    "Logistic Regression (L1)": LogisticRegression(
        max_iter=3000,
        class_weight="balanced",
        penalty="l1",
        solver="liblinear",
        random_state=42,
    ),
    "SGD Log-Loss": SGDClassifier(
        loss="log_loss",
        class_weight="balanced",
        max_iter=3000,
        tol=1e-4,
        random_state=42,
    ),
    "Random Forest": RandomForestClassifier(
        n_estimators=400,
        min_samples_leaf=2,
        class_weight="balanced",
        random_state=42,
    ),
}

model_results = []
fitted_models = {}

for name, model in models.items():
    model.fit(X_fit, y_fit)
    fitted_models[name] = model

    y_pred = model.predict(X_valid)
    if hasattr(model, "predict_proba"):
        y_score = model.predict_proba(X_valid)[:, 1]
```

```
    else:
        raw_score = model.decision_function(X_valid)
        y_score = 1 / (1 + np.exp(-raw_score))

    model_results.append(
        {
            "Model": name,
            "Accuracy": accuracy_score(y_valid, y_pred),
            "Precision": precision_score(y_valid, y_pred),
            "Recall": recall_score(y_valid, y_pred),
            "F1": f1_score(y_valid, y_pred),
            "ROC_AUC": roc_auc_score(y_valid, y_score),
        }
    )

metrics_df = pd.DataFrame(model_results).sort_values(
    by=["ROC_AUC", "F1"],
    ascending=False,
).reset_index(drop=True)
metrics_df.to_csv(Path.cwd() / "model_metrics.csv", index=False)

print("\nmodel metrics:")
print(metrics_df.round(4))


################################################################
# Part 13. Draw ROC curves and confusion matrices
################################################################

fig, ax = plt.subplots(figsize=(11, 8))

for name, model in fitted_models.items():
    if hasattr(model, "predict_proba"):
        y_score = model.predict_proba(X_valid)[:, 1]
    else:
        raw_score = model.decision_function(X_valid)
        y_score = 1 / (1 + np.exp(-raw_score))

    fpr, tpr, _ = roc_curve(y_valid, y_score)
    auc_value = roc_auc_score(y_valid, y_score)
```

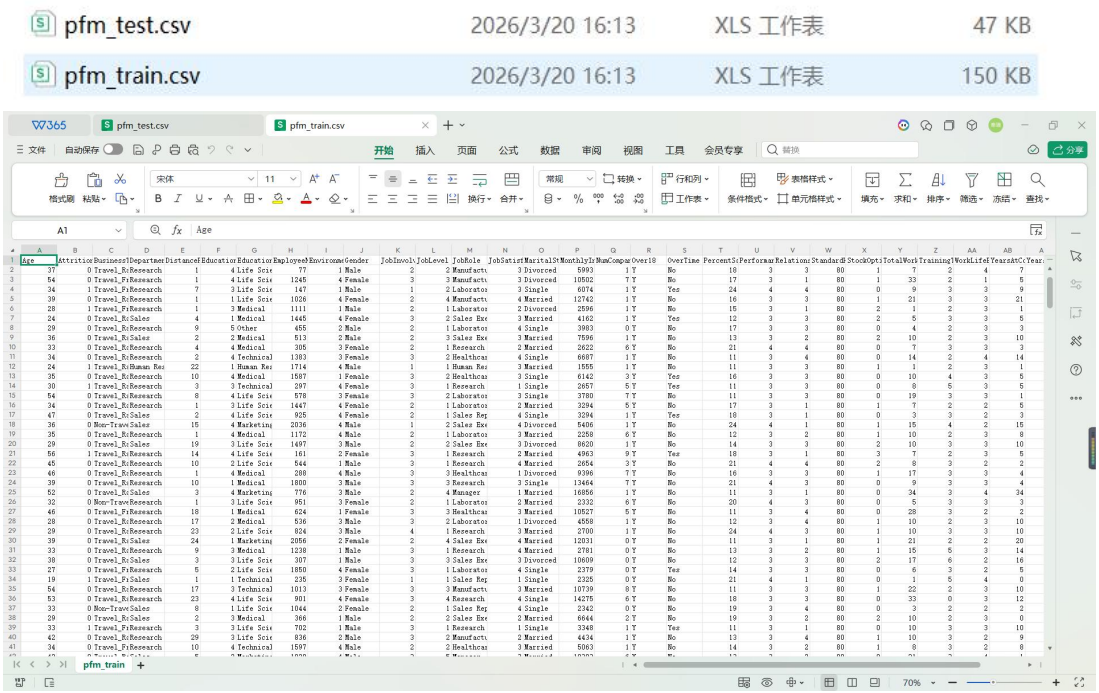
```
    ax.plot(fpr, tpr, linewidth=2.5, label=f"{name}
(AUC={auc_value:.3f})")

ax.plot([0, 1], [0, 1], linestyle="--", color="gray", linewidth=1.5,
label="Random Guess")
ax.set_title("ROC Curves of Different Models")
ax.set_xlabel("False Positive Rate")
ax.set_ylabel("True Positive Rate")
ax.legend(loc="lower right")
plt.tight_layout()
fig.savefig(figure_dir / "figure_04_roc_curves.png", dpi=200,
bbox_inches="tight")
plt.close(fig)

best_log_model = fitted_models["Logistic Regression (L2)"]
rf_model = fitted_models["Random Forest"]

cm_log = confusion_matrix(y_valid, best_log_model.predict(X_valid))
cm_rf = confusion_matrix(y_valid, rf_model.predict(X_valid))

fig, axes = plt.subplots(1, 2, figsize=(14, 6))

sns.heatmap(cm_log, annot=True, fmt="d", cmap="Blues", cbar=False,
ax=axes[0])
axes[0].set_title("Confusion Matrix: Logistic Regression (L2)")
axes[0].set_xlabel("Predicted Label")
axes[0].set_ylabel("True Label")

sns.heatmap(cm_rf, annot=True, fmt="d", cmap="Greens", cbar=False,
ax=axes[1])
axes[1].set_title("Confusion Matrix: Random Forest")
axes[1].set_xlabel("Predicted Label")
axes[1].set_ylabel("True Label")

plt.tight_layout()
fig.savefig(figure_dir / "figure_05_confusion_matrices.png", dpi=200,
bbox_inches="tight")
plt.close(fig)


################################################################
```

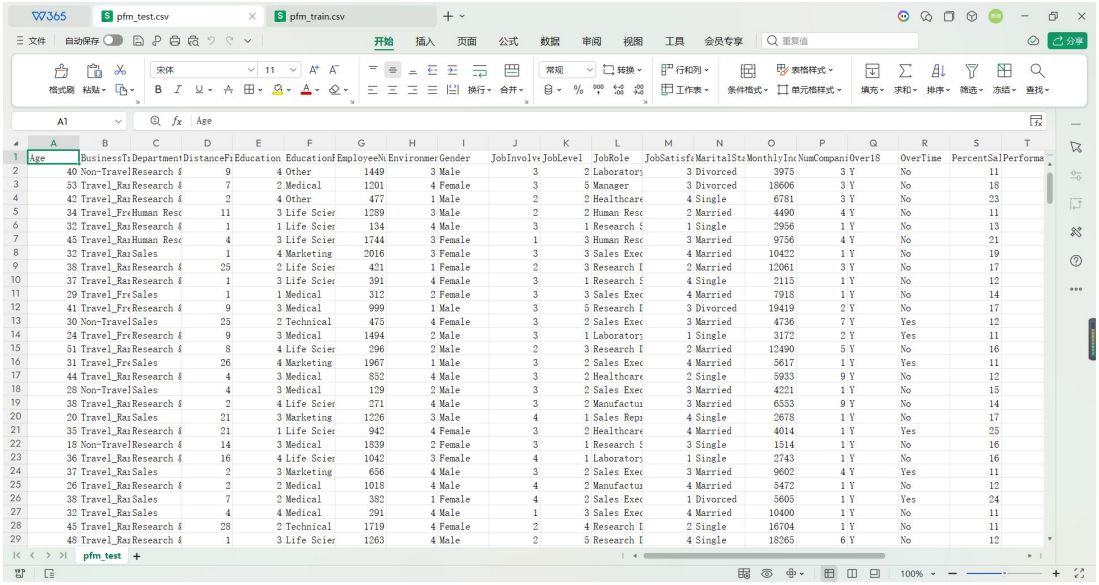
```
# Part 14. Save coefficient and feature importance outputs
################################################################

log_coef = pd.Series(best_log_model.coef_[0],
index=X_train.columns).sort_values()
top_negative = log_coef.head(8).sort_values(ascending=True)
top_positive = log_coef.tail(8).sort_values(ascending=False)

rf_importance = (
    pd.Series(rf_model.feature_importances_, index=X_train.columns)
    .sort_values(ascending=False)
    .head(12)
)

top_positive.to_csv(Path.cwd() /
"logistic_top_positive_coefficients.csv", header=["coefficient"])
top_negative.to_csv(Path.cwd() /
"logistic_top_negative_coefficients.csv", header=["coefficient"])
rf_importance.to_csv(Path.cwd() /
"random_forest_feature_importances.csv", header=["importance"])

fig, axes = plt.subplots(1, 2, figsize=(18, 8))

coef_plot = pd.concat([top_positive, top_negative]).sort_values()
sns.barplot(
    x=coef_plot.values,
    y=coef_plot.index,
    palette=["#4c78a8" if value < 0 else "#e45756" for value in
coef_plot.values],
    ax=axes[0],
)
axes[0].set_title("Most Influential Logistic Regression Coefficients")
axes[0].set_xlabel("Coefficient Value")
axes[0].set_ylabel("Feature")

sns.barplot(
    x=rf_importance.values,
    y=rf_importance.index,
    palette="rocket",
    ax=axes[1],
```

```
)
axes[1].set_title("Top Random Forest Feature Importances")
axes[1].set_xlabel("Importance")
axes[1].set_ylabel("Feature")

plt.tight_layout()
fig.savefig(figure_dir / "figure_06_model_interpretation.png", dpi=200,
bbox_inches="tight")
plt.close(fig)

print("\ntop positive logistic coefficients:")
print(top_positive)
print("\ntop negative logistic coefficients:")
print(top_negative)
```

4. Experimental Data and Results

Experimental Data:





Experimental Results:

```
train shape: (1100, 31)
test shape: (350, 30)
```

	Age	Attrition	BusinessTravel	Department	DistanceFromHome	Education	EducationField	EmployeeNumber	EnvironmentSatisfaction	Gender	...	RelationshipSatisfaction
0	37	0	Travel_Rarely	Research & Development	1	4	Life Sciences	77	1	Male	...	3
1	54	0	Travel_Frequently	Research & Development	1	4	Life Sciences	1245	4	Female	...	1
2	34	1	Travel_Frequently	Research & Development	7	3	Life Sciences	147	1	Male	...	4
3	39	0	Travel_Rarely	Research & Development	1	1	Life Sciences	1026	4	Female	...	3
4	28	1	Travel_Frequently	Research & Development	1	3	Medical	1111	1	Male	...	1

5 rows × 31 columns

```
=== train info ===
<class 'pandas.DataFrame'>
RangeIndex: 1100 entries, 0 to 1099
Data columns (total 31 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Age                       1100 non-null   int64
 1   Attrition                 1100 non-null   int64
 2   BusinessTravel            1100 non-null   str
 3   Department                1100 non-null   str
 4   DistanceFromHome          1100 non-null   int64
 5   Education                 1100 non-null   int64
 6   EducationField            1100 non-null   str
 7   EmployeeNumber            1100 non-null   int64
 8   EnvironmentSatisfaction   1100 non-null   int64
 9   Gender                    1100 non-null   str
 10  JobInvolvement            1100 non-null   int64
 11  JobLevel                  1100 non-null   int64
 12  JobRole                   1100 non-null   str
 13  JobSatisfaction           1100 non-null   int64
 14  MaritalStatus             1100 non-null   str
 15  MonthlyIncome             1100 non-null   int64
 16  NumCompaniesWorked        1100 non-null   int64
 17  Over18                    1100 non-null   str
 18  OverTime                  1100 non-null   str
...
YearsInCurrentRole          0
YearsSinceLastPromotion     0
YearsWithCurrManager        0
dtype: int64
```

Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output

```
duplicate rows in train: 0
duplicate rows in test: 0
constant cols in train: ['Over18', 'StandardHours']
constant cols in test: ['Over18', 'StandardHours']
```

```
attrition counts:
Attrition
0    922
1    178
Name: count, dtype: int64

raw categorical cols: ['BusinessTravel', 'Department', 'EducationField', 'Gender', 'JobRole', 'MaritalStatus', 'Over18', 'OverTime']

value counts of BusinessTravel:
BusinessTravel
Travel_Rarely        787
Travel_Frequently    205
Non-Travel           108
Name: count, dtype: int64

value counts of Department:
Department
Research & Development    727
Sales                     331
Human Resources            42
Name: count, dtype: int64

value counts of EducationField:
EducationField
Life Sciences       462
...
OverTime
No     794
Yes    306
Name: count, dtype: int64
```

Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...

```
C:\Users\rober\AppData\Local\Temp\ipykernel_30896\1742703896.py:6: Pandas4Warning: For backward compatibility, 'str' dtypes are includ
```

```
train_clean shape: (1100, 28)
test_clean shape: (350, 27)
```

```
categorical cols after drop: ['BusinessTravel', 'Department', 'EducationField', 'Gender', 'JobRole', 'MaritalStatus', 'OverTime']
encoded train shape: (1100, 42)
encoded test shape: (350, 41)
X_train shape: (1100, 41)
X_test shape: (350, 41)
C:\Users\rober\AppData\Local\Temp\ipykernel_30896\545202335.py:2: Pandas4Warning: For backward compatibility, 'str' dtypes are included by select_dtypes when 'object
See https://pandas.pydata.org/docs/user_guide/migration-3-strings.html#string-migration-select-dtypes for details on how to write code that works with pandas 2 and 3
  cat_cols = train_clean.select_dtypes(include="object").columns.tolist()
```

```
scaled numeric cols: ['Age', 'DistanceFromHome', 'Education', 'EnvironmentSatisfaction', 'JobInvolvement', 'JobLevel', 'JobSatisfaction', 'MonthlyIncome', 'NumCompanie
```

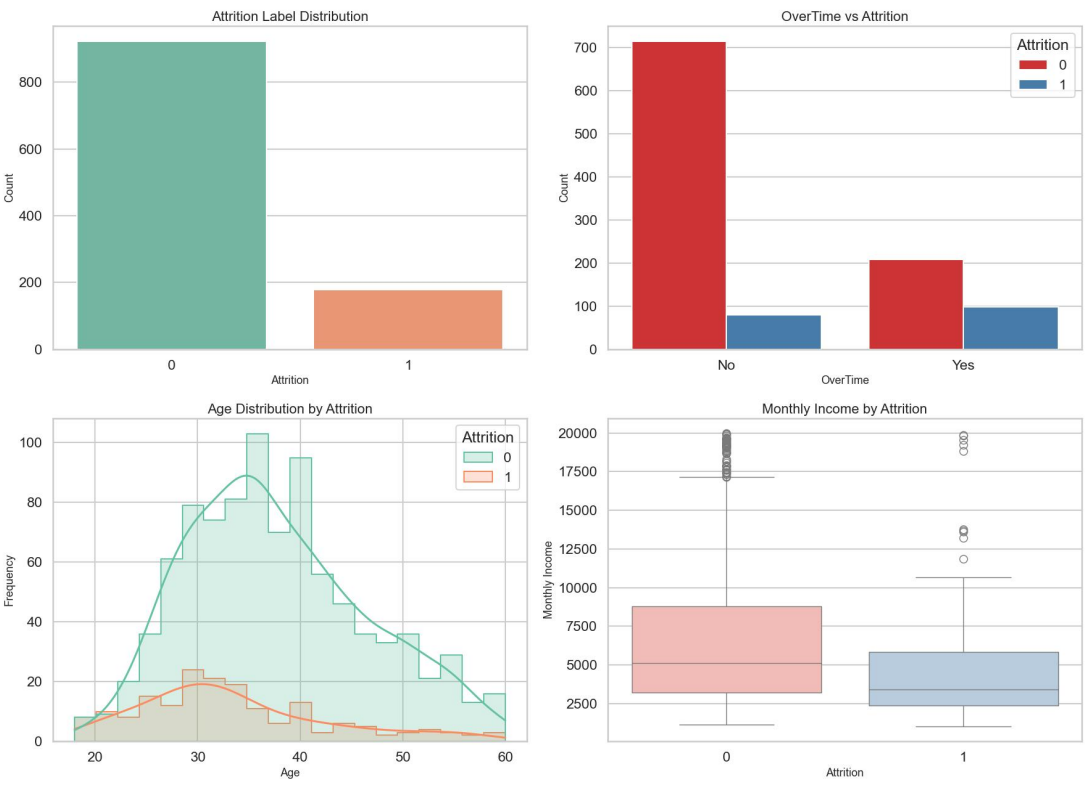
	Age	DistanceFromHome	Education	EnvironmentSatisfaction	JobInvolvement	JobLevel	JobSatisfaction	MonthlyIncome	NumCompaniesWorked	PercentSalaryHike	...
0	0.000101	-1.028598	1.054313	-1.572091	-1.035216	-0.049260	0.240954	-0.104096	-0.671072	0.762229	...
1	1.882064	-1.028598	1.054313	1.161260	0.381124	0.853837	0.240954	0.852589	1.720437	0.486513	...
2	-0.332010	-0.296263	0.075626	-1.572091	-2.451557	-0.049260	0.240954	-0.086910	-0.671072	2.416525	...
3	0.221508	-1.028598	-1.881748	1.161260	-1.035216	1.756934	1.142483	1.327855	-0.671072	0.210797	...
4	-0.996233	-1.028598	0.075626	-1.572091	-1.035216	-0.952357	-0.660575	-0.824846	-0.671072	-0.064919	...

5 rows × 41 columns

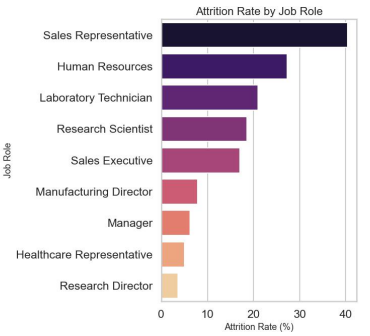
```
final X_train shape: (1100, 41)
final y_train shape: (1100,)
final X_test shape: (350, 41)
missing values in X_train: 0
missing values in X_test: 0

X_train dtype summary:
int64      21
float64    20
Name: count, dtype: int64
```

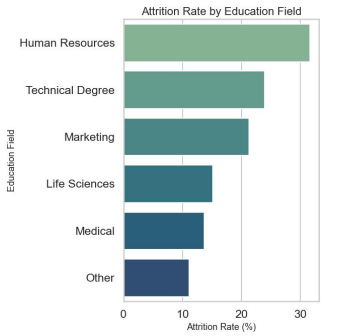
线性模型+实践.pptx	2026/4/10 16:03	PPTX 演示文稿	64 KB
~$线性模型+实践.pptx	2026/4/17 15:55	PPTX 演示文稿	1 KB
X_train_preprocessed.csv	2026/4/17 16:35	XLS 工作表	470 KB
y_train_preprocessed.csv	2026/4/17 16:35	XLS 工作表	4 KB
X_test_preprocessed.csv	2026/4/17 16:35	XLS 工作表	150 KB

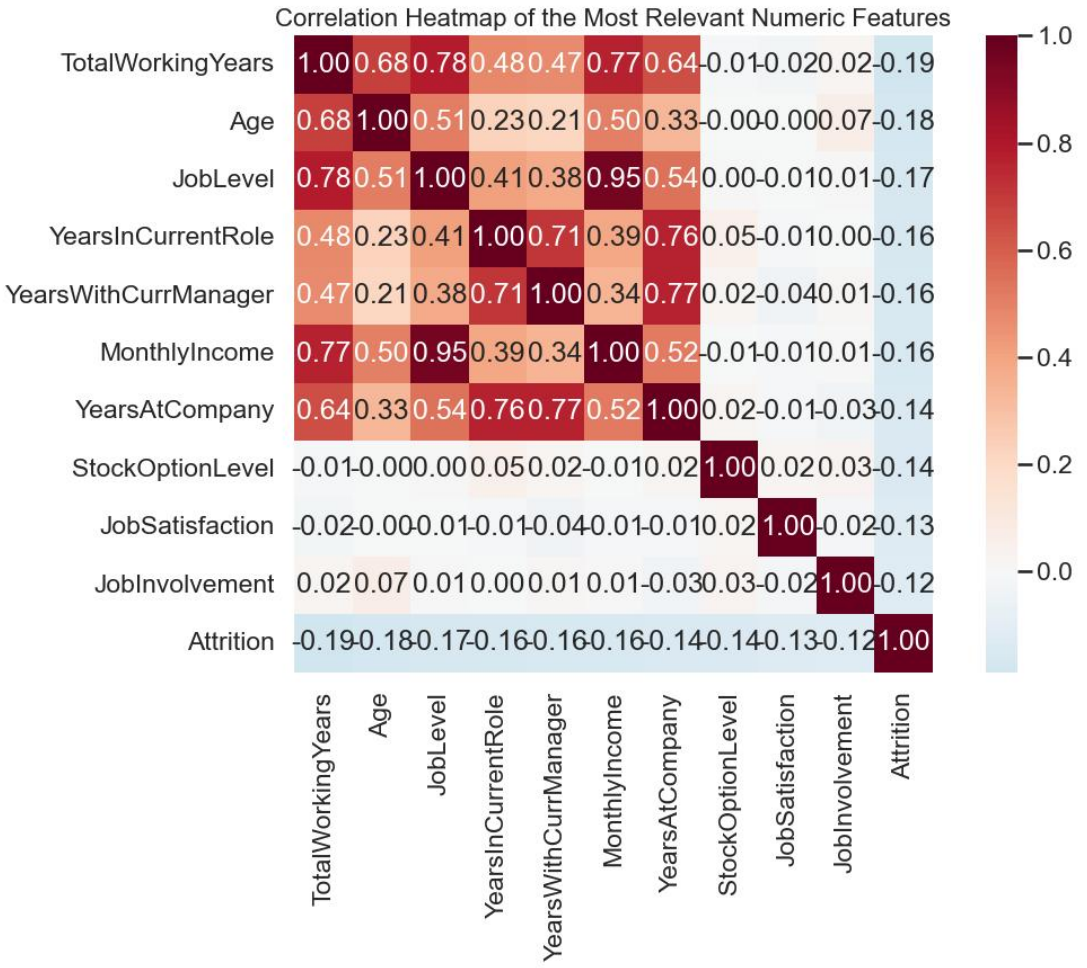
```
prepared_X shape: (1100, 41)
prepared_y shape: (1100,)
X_fit shape: (825, 41)
X_valid shape: (275, 41)
validation label distribution:
Attrition
0    0.84
1    0.16
Name: proportion, dtype: float64
```




Attrition Rate by Department
Human Resources
Sales
Research & Development
Department
0
5
10
15
20
Attrition Rate (%)

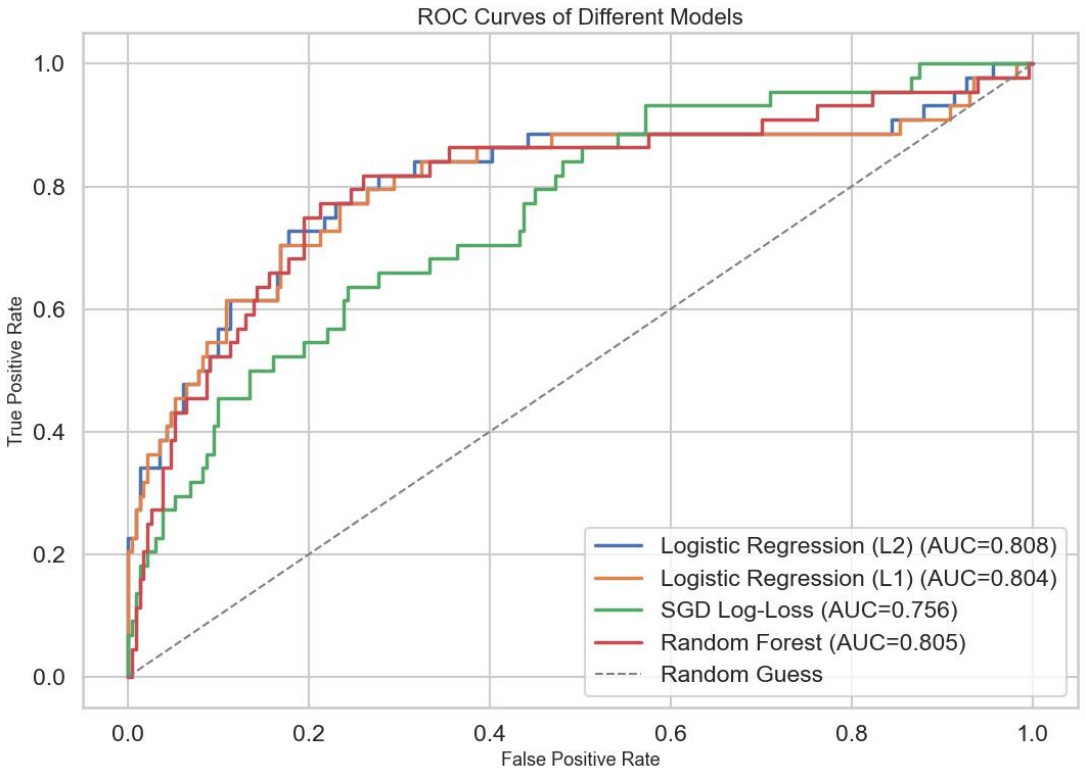

Attrition Rate by Job Role
Sales Representative
Human Resources
Laboratory Technician
Research Scientist
Sales Executive
Manufacturing Director
Manager
Healthcare Representative
Research Director
Job Role
0
10
20
30
40
Attrition Rate (%)

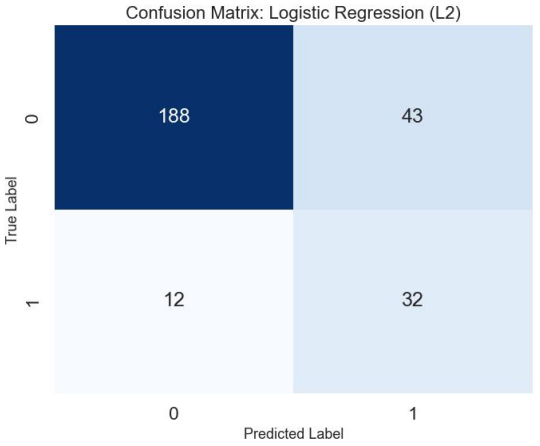

Attrition Rate by Education Field
Human Resources
Technical Degree
Marketing
Life Sciences
Medical
Other
Education Field
0
10
20
30
Attrition Rate (%)

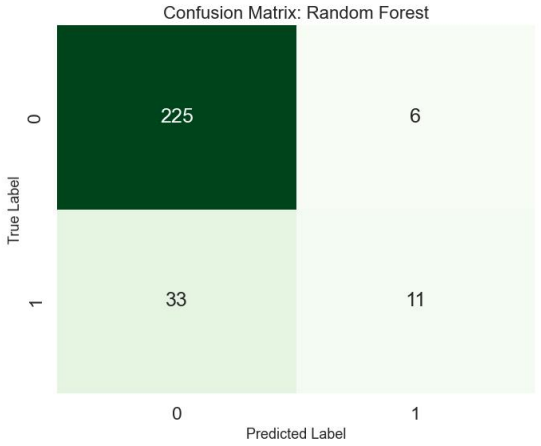


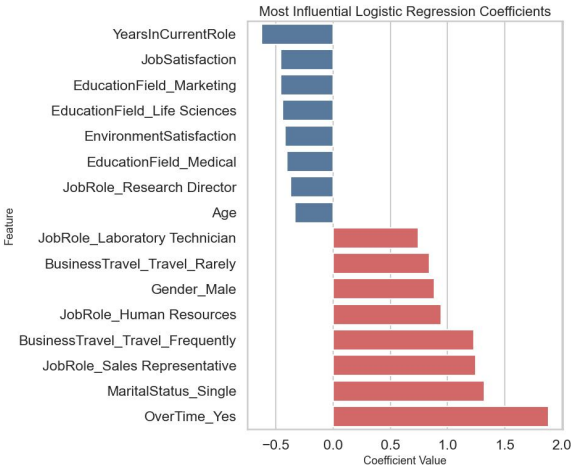

```
Top numeric correlations with Attrition:
TotalWorkingYears       -0.187922
Age                     -0.175393
JobLevel                -0.168775
YearsInCurrentRole      -0.163059
YearsWithCurrManager    -0.158558
MonthlyIncome           -0.155521
YearsAtCompany          -0.143697
StockOptionLevel        -0.138498
JobSatisfaction         -0.125568
JobInvolvement          -0.122722
Name: Attrition, dtype: float64
```

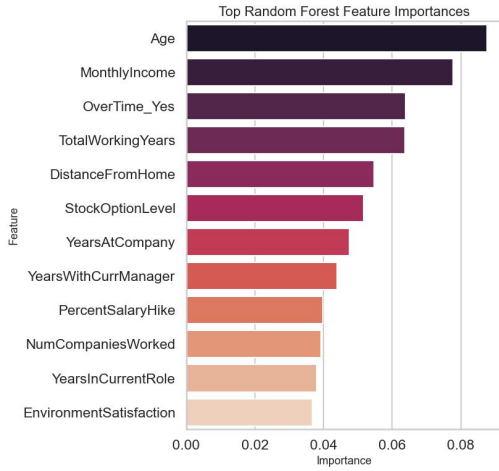
	Model	Accuracy	Precision	Recall	F1	ROC_AUC
0	Logistic Regression (L2)	0.8000	0.4267	0.7273	0.5378	0.8081
1	Random Forest	0.8582	0.6471	0.2500	0.3607	0.8051
2	Logistic Regression (L1)	0.7818	0.3974	0.7045	0.5082	0.8044
3	SGD Log-Loss	0.7455	0.3289	0.5682	0.4167	0.7559












```
Top positive logistic coefficients:
OverTime_Yes                        1.881668
MaritalStatus_Single                1.319553
JobRole_Sales Representative        1.247747
BusinessTravel_Travel_Frequently    1.231700
JobRole_Human Resources             0.943837
Gender_Male                         0.883100
BusinessTravel_Travel_Rarely        0.841504
JobRole_Laboratory Technician       0.743538
dtype: float64

Top negative logistic coefficients:
YearsInCurrentRole            -0.627042
JobSatisfaction               -0.457929
EducationField_Marketing      -0.457115
EducationField_Life Sciences  -0.444256
EnvironmentSatisfaction       -0.420768
EducationField_Medical        -0.403998
JobRole_Research Director     -0.373545
Age                           -0.335217
dtype: float64
```

experiment1_figures	2026/4/17 16:35	文件夹	
_ppt_copy.pptx	2026/3/20 16:13	PPTX 演示文稿	139 KB
experiment1_preprocessing_full.py	2026/4/17 16:31	PY 文件	17 KB
logistic_top_negative_coefficients.csv	2026/4/17 16:35	XLS 工作表	1 KB
logistic_top_positive_coefficients.csv	2026/4/17 16:35	XLS 工作表	1 KB
model_metrics.csv	2026/4/17 16:35	XLS 工作表	1 KB
pfm_test.csv	2026/3/20 16:13	XLS 工作表	47 KB
pfm_train.csv	2026/3/20 16:13	XLS 工作表	150 KB
random_forest_feature_importance...	2026/4/17 16:35	XLS 工作表	1 KB

5. Reflections on the Experiment

This experiment gave me a much stronger feeling that **data analysis** is never just about **"choosing a model and pressing run."** At the beginning, I thought the main work would probably be **filling missing values**, because that is usually the first problem mentioned in **data cleaning**. But after I checked the dataset with **info()** and **isnull().sum()**, I found that there were actually **no missing values** in the training and test sets. That result surprised me a little. It made me realize that **real problems** are not always the most obvious ones. In this dataset, the more important issues were **constant columns**, **identifier columns**, **categorical encoding**, and keeping the **train and test feature spaces consistent**.

One problem that took me some time to understand was the role of **categorical features** in **linear models**. At first, those columns just looked like ordinary **text fields**, but once I started thinking about **model input**, I realized they could not be used directly at all. My first concern was whether converting them would change the **meaning of the data** too much. Later, after comparing the different categories and seeing how many important **employee attributes** were stored as text, I understood that dropping them would actually lose too much **information**. That is why **one-hot encoding** became necessary. But after solving that problem, another one appeared immediately: the encoded training set and the encoded test set might not end up with exactly the same **columns**. I found that this was one of those details that looks very small in code but is actually very serious inpractice. Using **reindex** was the step that really

fixed this issue, and after I understood it, I felt I had learned something much more practical than just a **syntax trick**.

Another thing I learned from this experiment was that **preprocessing decisions** directly affect **model behavior** later. For example, I standardized the **numeric features** because I knew that **linear models** are sensitive to **scale**. Before doing this experiment, I only understood **"standardization"** as a general **textbook concept**. But after actually writing the code, I understood why the **order matters**: the scaler must be fitted on the **training data** first, and then the same rule must be applied to **validation** or **test data**. Otherwise, the whole process becomes loose and can even introduce **information leakage**. This made me more careful with the distinction between **fit_transform()** and **transform()**. It is a small coding detail, but it carries an important **methodological meaning**.

The modeling stage also gave me a more realistic view of **evaluation**. At first, it was tempting to look mainly at **accuracy**, because accuracy is the easiest number to read. But once I checked the **label distribution**, I found that the **attrition class** was much smaller than the **non-attrition class**. That meant a model could still get a decent **accuracy** even if it missed many employees who were actually likely to leave. Because of that, I started paying much more attention to **recall**, **F1-score**, and **ROC-AUC**. This changed how I looked at the results. For example, a model with higher **accuracy** was not always the best choice if its **recall** was too low. That made the comparison between **logistic regression** and **random forest** much more meaningful, and it also helped me understand why a **linear model** can still be a strong and practical choice even if it is not the most complicated model in the experiment.

I also ran into a very concrete **workflow problem** during the experiment: some generated **output files** had been deleted, and then the old **export logic** failed because it was trying to infer the **output directory** by checking whether certain files already existed. At first this error looked confusing, because the **preprocessing code** itself was correct, but the export step still stopped. After tracing the problem, I realized the issue was not the **data transformation logic**, but the **path selection logic**. The fix was to stop depending on previously existing files and instead write the outputs directly to a **clear target folder**. This small **debugging experience** made the experiment feel more real to me, because

it reminded me that practical **data work** is not only about **algorithms**. **File organization**, **reproducibility**, and **stable code structure** are also part of doing the job well.

Overall, this experiment helped me connect many things that used to feel separate in theory: **raw data inspection**, **cleaning**, **feature encoding**, **scaling**, **train-test alignment**, **model comparison**, **metric selection**, and **result interpretation**. What I gained most was not just a set of **processed csv files** or a finished **notebook**, but a clearer sense of how one step leads to the next. When a later model works well, it is often because the earlier **preprocessing** was done carefully. When a result looks strange, the reason may be hidden several steps earlier. That is probably the biggest lesson I learned here: a good **model result** usually begins with **patient**, **detailed**, and sometimes even **tedious preprocessing work**.

Grade:

Reviewed by: --------

Date: --------